

DOKUMENTACJA TECHNICZNA PROJEKTU

SYSTEM ALARMOWY ADUINO

Tomasz Krzysiak – 21477 – II rok – II sem. – SN – INFORMATYKA

1. Ogólny opis projektu

Projekt przedstawia średnio-zaawansowany, wielostrefowy system alarmowy oparty na mikrokontrolerze Arduino Uno. Centrala integruje różnorodne strefy detekcji (czujnik ruchu PIR, przełączniki obwodowe reprezentujące okna i drzwi, oraz przycisk wibracyjny) wraz z wielopoziomowym interfejsem użytkownika. Interfejs składa się z ekranu LCD obsługiwanego za pomocą dedykowanego menu, panelu wizualizacji stanów opartego na adresowalnym pasku LED NeoPixel, sygnalizatorów optyczno-akustycznych (buzzer i diody LED) oraz zewnętrznego wyświetlacza 7-segmentowego informującego na bieżąco o aktualnym stanie centrali. Całość zarządzana jest przez zoptymalizowany algorytm maszyny stanów (State Machine), co gwarantuje stabilne i asynchroniczne działanie systemu bez blokowania pętli głównej programu.

2. Architektura systemu

Sercem oprogramowania układowego jest struktura maszyny stanów, która przyjmuje 5 ściśle zdefiniowanych trybów pracy. Każdy stan jest w czasie rzeczywistym komunikowany na wyświetlaczu LCD, pasku diod adresowalnych NeoPixel (kolor oraz pozycja diody) oraz na wyświetlaczu 7-segmentowym poprzez dekodery CD4511:

Nr Stanu	Nazwa systemowa	Opis stanu centrali	Wizualizacja NeoPixel	Cyfra 7-SEG
0	ALARM_DISARMED	System rozbrojony, brak nadzoru linii czujników.	Dioda 0: Zielony	0
1	ALARM_ARMED	System uzbrojony, aktywne pełne czuwanie stref.	Dioda 1: Niebieski	1
2	ALARM_EXIT_DELAY	Odliczanie czasu na bezpieczne opuszczenie obiektu (10s).	Dioda 2: Pomarańczowy	2

3	ALARM_TRIGGERED	Wykrycie intruza, faza głośnego i optycznego alarmu.	Dioda 3: Czerwony	3
4	SENSOR_VIOLATED	Tryb diagnostyczny (test czujników wywoływany z menu).	Dioda 4: Fioletowy	4

3. Specyfikacja sprzętowa

Poniższa tabela przedstawia kompletny opis mapowania pinów mikrokontrolera Arduino Uno R3, realizujący architekturę sprzętową projektu:

Pin Arduino	Typ we/wy	Podłączone urządzenie	Rola w systemie i opis logiczny
0	OUTPUT	CD4511 – WEJŚCIE 1 (A)	Sterowanie wyświetlaczem 7-SEG (bit najmłodszy LSB)
1	OUTPUT	CD4511 – WEJŚCIE 2 (B)	Sterowanie wyświetlaczem 7-SEG (bit 1)
2	INPUT	Czujnik ruchu PIR	Cyfrowa strefa alarmowa (wymaga potrójnego wzbudzenia)
3	OUTPUT	LCD – DB7	Linia danych magistrali 4-bitowej wyświetlacza LCD
4	OUTPUT	LCD – DB6	Linia danych magistrali 4-bitowej wyświetlacza LCD
5	OUTPUT	LCD – DB5	Linia danych magistrali 4-bitowej wyświetlacza LCD
6	OUTPUT	LCD – DB4	Linia danych magistrali 4-bitowej wyświetlacza LCD

7	INPUT_PULLUP	Przełącznik strefowy	Czujnik wibracyjny (wywołuje natychmiastowy alarm)
8	OUTPUT	Dioda LED Niebieska	Optyczna sygnalizacja alarmowa (kogut policyjny)
9	OUTPUT	Dioda LED Czerwona	Optyczna sygnalizacja alarmowa (kogut policyjny)
10	INPUT_PULLUP	Przycisk fizyczny (UP)	Nawigacja interfejsu LCD (przewijanie menu w górę)
11	OUTPUT	LCD – E (Enable)	Linia strobulująca (zegarowa) wyświetlacza LCD
12	OUTPUT	LCD – RS (Register Select)	Wybór rejestru instrukcji lub danych wyświetlacza LCD
13	OUTPUT	CD4511 – WEJŚCIE 3 (C)	Sterowanie wyświetlaczem 7-SEG (bit 2)
A0	INPUT_PULLUP	Przycisk fizyczny (DOWN)	Nawigacja interfejsu LCD (przewijanie menu w dół)
A1	INPUT_PULLUP	Przycisk fizyczny (OK)	Zatwierdzanie wybranej opcji w menu konfiguracyjnym
A2	OUTPUT	Pasek NeoPixel (DIN)	Jednoprzewodowa cyfrowa magistrala sterująca panelem LED
A3	INPUT_PULLUP	Przełącznik strefowy	Czujnik magnetyczny okna (natychmiastowe wzbudzenie)

A4	OUTPUT	Buzzer Piezo	Generator sygnałów dźwiękowych, odliczania i modulowanej syreny
A5	INPUT_PULLUP	Przełącznik strefowy	Czujnik magnetyczny drzwi (natychmiastowe wzbudzenie)

Ważna optymalizacja układowa: Wejście 4 (D) układu CD4511 zostało na stałe połączony z szyną masy (GND). Ponieważ stany systemu alarmowego mieszczą się ściśle w zakresie od 0 do 4, najbardziej znaczący bit (MSB) reprezentacji binarnej tych liczb zawsze przyjmuje wartość 0. Taki zabieg pozwolił zaoszczędzić jeden cenny pin cyfrowy Arduino, zapobiegając konfliktom sprzętowym z liniami magistrali wyświetlacza LCD.

4. Algorytm i logika działania programu

- **Dynamiczne dodawanie czujników:** Zaimplementowany mechanizm pozwala na rozszerzanie systemu o nowe strefy w czasie rzeczywistym z poziomu interfejsu użytkownika. Nowo zadeklarowany pin (cyfrowy lub analogowy działający w trybie cyfrowym) jest dynamicznie dodawany do tablicy struktur typu Sensor, konfigurując wejście z wewnętrznym podciągnięciem (INPUT_PULLUP) bez potrzeby resetowania urządzenia.
- **Algorytm filtracji czujnika PIR:** W celu eliminacji fałszywych alarmów wywoływanych przez chwilowe zakłócenia otoczenia, czujnik ruchu (PIR) na pinie 2 nie wywołuje alarmu natychmiast. Dopiero potrójne (3) wykrycie impulsu w stanie czuwania przełącza system w tryb krytyczny (ALARM_TRIGGERED). Pozostałe czujniki obwodowe (drzwi, okna, wibracje) działają natychmiastowo.
- **Automatyczny Timeout Menu:** Bezpieczeństwo systemu zwiększa funkcja monitorująca aktywność użytkownika w menu. Jeśli centrala zostanie pozostawiona w trybie konfiguracji lub dodawania czujników przez czas dłuższy niż 12 sekund bez żadnej reakcji (naciśnięcia przycisku), następuje automatyczny powrót do ekranu głównego.
- **Asynchroniczna sygnalizacja akustyczno-optyczna:** W stanie wyzwolenia alarmu centrala generuje dwutonową syrenę modulowaną oraz asynchroniczne, naprzemienne miganie diody czerwonej i niebieskiej. Dzięki oparciu algorytmu o funkcję millis(), cała sygnalizacja przebiega całkowicie bezbłokowo.

5. Instrukcja obsługi system

1. **Inicjalizacja:** Po podłączeniu zasilania system przez 2 sekundy wyświetla ekran powitalny, przeprowadza autodiagnostykę i przechodzi do stanu bezpiecznego 'System OFF' (cyfra 0 na wyświetlaczu 7-SEG, zielony dioda na NeoPixel).
2. **Uzbrojenie:** Naciśnij przycisk UP lub DOWN, aby wejść do menu. Wybierz opcję 'Uzbroj alarm' i zatwierdź przyciskiem OK. System uruchomi 10-sekundową fazę na opuszczenie chronionego obiektu (buzzer pika sekwencyjnie, wyświetlacz 7-SEG wskazuje cyfrę 2). Po odliczeniu czasu centrala przechodzi w stan pełnego czuwania (cyfra 1 na 7-SEG, niebieska dioda NeoPixel).
3. **Rozbrojenie:** Aby wyłączyć czuwanie lub uciszyć aktywny alarm, wejdź w menu przyciskami nawigacyjnymi, wybierz opcję 'Rozbroj alarm' i zatwierdź przyciskiem OK. System natychmiast powraca do stanu bezpiecznego.
4. **Tryb testowy:** Wejdź do menu i wybierz opcję 'Test czujników', potwierdzając przyciskiem OK. Centrala przejdzie w tryb niegroźnego sprawdzania stref (wyświetlacz 7-SEG wskazuje cyfrę 4, NeoPixel świeci na fioletowo). Pobudzenie dowolnego sensora pokaże na ekranie LCD znacznik '[!]', pozwalając sprawdzić sprawność instalacji bez uruchamiania syreny.

6. Pełny kod źródłowy programu

```
#include <Adafruit_NeoPixel.h>

// KONFIGURACJA PINÓW
const int ledBlue = 8; const int ledRed = 9;
const int btnUP = 10; const int btnDOWN = A0; const int btnOK = A1;
const int PIN_NEOPIXEL = A2;
const int PIN_BUZZER = A4;
const int NUMPIXELS = 8;
Adafruit_NeoPixel panelLED(NUMPIXELS, PIN_NEOPIXEL, NEO_GRB + NEO_KHZ800);

// PINY DEKODERA CD4511
const int PIN_A = 0; // WEJSCIE 1 w CD4511
const int PIN_B = 1; // WEJSCIE 2 w CD4511
const int PIN_C = 13; // WEJSCIE 3 w CD4511
// WEJSCIE 4 w CD4511 jest na stałe podłączone do masy (GND)

const int RS = 12; const int E = 11;
const int DB4 = 6; const int DB5 = 5; const int DB6 = 4; const int DB7 = 3;

// STRUKTURY I TYPY
#define SENSOR_MAX 6
#define SENSOR_PIR 1
#define SENSOR_SWITCH 2

struct Sensor {
    byte pin;
    byte type;
    bool violated;
    int lastState;
};

Sensor sensors[SENSOR_MAX];
int sensorCount = 0;

// STANY SYSTEMU
#define ALARM_DISARMED 0
#define ALARM_ARMED 1
#define ALARM_EXIT_DELAY 2
#define ALARM_TRIGGERED 3
#define SENSOR_VIOLATED 4

int systemState = ALARM_DISARMED;
int alarm_sense = 0;
int currentMenuPos = 0;
bool inMenu = false;
bool inSensorList = false;
bool inAddMode = false;
int tempPin = 4;

int currentSensorIdx = 0;
unsigned long lastActivityTime = 0;
unsigned long exitTimer = 0;
int countdownValue = 0;
```

```

int lastUp = HIGH; int lastDown = HIGH; int lastOk = HIGH;
unsigned long prevMillis = 0; bool blink = false;

// DEKLARACJE FUNKCJI
void checkSensors(); void handleButtons(); void displayMenu(); void
displaySensorList();
void displayMainScreen(); void updateVisuals(); void executeAction(); void
LCD_Init4bit();
void LCD_WriteCommand4Bits(byte d); void LCD_WriteData4Bits(byte d); void
LCD_WriteText(const char *s);
void displayAddScreen(); void addSensor(byte pin, byte type);
void update7SegmentCD4511(int state);

void addSensor(byte pin, byte type) {
    if (sensorCount < SENSOR_MAX) {
        sensors[sensorCount].pin = pin;
        sensors[sensorCount].type = type;
        sensors[sensorCount].violated = false;

        if (pin == 2) {
            pinMode(pin, INPUT);
        } else {
            pinMode(pin, INPUT_PULLUP);
        }

        sensors[sensorCount].lastState = digitalRead(pin);
        sensorCount++;
    }
}

void setup() {
    panelLED.begin();
    pinMode(ledBlue, OUTPUT); pinMode(ledRed, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    pinMode(btnUP, INPUT_PULLUP); pinMode(btnDOWN, INPUT_PULLUP); pinMode(btnOK,
INPUT_PULLUP);

    // Ustawienie pinów sterujących dekodera jako wyjścia
    pinMode(PIN_A, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    pinMode(PIN_C, OUTPUT);

    pinMode(E, OUTPUT); pinMode(RS, OUTPUT);
    pinMode(DB4, OUTPUT); pinMode(DB5, OUTPUT); pinMode(DB6, OUTPUT); pinMode(DB7,
OUTPUT);

    LCD_Init4bit();

    // Czujniki bazowe
    addSensor(2, SENSOR_PIR);
    addSensor(A3, SENSOR_SWITCH);
    addSensor(A5, SENSOR_SWITCH);
    addSensor(7, SENSOR_SWITCH);

    LCD_WriteCommand4Bits(0x01);
    LCD_WriteCommand4Bits(0x80); LCD_WriteText("SYSTEM ALARMOWY ");
    LCD_WriteCommand4Bits(0xC0); LCD_WriteText("START... ");
    delay(2000);

```

```

}

void loop() {
    checkSensors();
    handleButtons();

    if (systemState == ALARM_EXIT_DELAY) {
        if (millis() - exitTimer >= 1000) {
            exitTimer = millis();
            countdownValue--;
            tone(PIN_BUZZER, 1000, 50);
            if (countdownValue <= 0) {
                systemState = ALARM_ARMED;
                alarm_sense = 0;
            }
        }
    }

    if (inAddMode) displayAddScreen();
    else if (inMenu) displayMenu();
    else if (inSensorList) displaySensorList();
    else displayMainScreen();

    updateVisuals();
}

void checkSensors() {
    bool anyViolated = false;
    for (int i = 0; i < sensorCount; i++) {
        int val = digitalRead(sensors[i].pin);
        bool isCurrentlyViolated = (val == HIGH);

        if (sensors[i].lastState == LOW && val == HIGH) {
            if (systemState == ALARM_ARMED) {
                if (sensors[i].pin == 2) {
                    alarm_sense++;
                    if (alarm_sense >= 3) systemState = ALARM_TRIGGERED;
                } else {
                    systemState = ALARM_TRIGGERED;
                }
            }
        }

        sensors[i].violated = isCurrentlyViolated;
        sensors[i].lastState = val;
        if (sensors[i].violated) anyViolated = true;
    }

    if (inSensorList && anyViolated) systemState = SENSOR_VIOLATED;
    else if (inSensorList) systemState = ALARM_DISARMED;
}

void handleButtons() {
    int u = digitalRead(btnUP); int d = digitalRead(btnDOWN); int o =
digitalRead(btnOK);

    if (lastUp == HIGH && u == LOW) {
        lastActivityTime = millis();
    }
}

```



```

        if (inAddMode) { tempPin++; if (tempPin > 19) tempPin = 2; }
        else if (inSensorList) currentSensorIdx = (currentSensorIdx <= 0) ? sensorCount -
1 : currentSensorIdx - 1;
        else { inMenu = true; currentMenuPos = (currentMenuPos <= 0) ? 3 : currentMenuPos
- 1; }
        LCD_WriteCommand4Bits(0x01);
    }

    if (lastDown == HIGH && d == LOW) {
        lastActivityTime = millis();
        if (inAddMode) { tempPin--; if (tempPin < 2) tempPin = 19; }
        else if (inSensorList) currentSensorIdx = (currentSensorIdx >= sensorCount - 1) ?
0 : currentSensorIdx + 1;
        else { inMenu = true; currentMenuPos = (currentMenuPos >= 3) ? 0 : currentMenuPos
+ 1; }
        LCD_WriteCommand4Bits(0x01);
    }

    if (lastOk == HIGH && o == LOW) {
        lastActivityTime = millis();
        if (inAddMode) {
            addSensor(tempPin, SENSOR_SWITCH);
            inAddMode = false; inMenu = true;
            LCD_WriteCommand4Bits(0x01); LCD_WriteText("DODANO CZUJNIK!"); delay(1000);
        }
        else if (inSensorList) { inSensorList = false; inMenu = true; }
        else executeAction();
        LCD_WriteCommand4Bits(0x01);
    }

    lastUp = u; lastDown = d; lastOk = o;

    if ((inMenu || inSensorList || inAddMode) && (millis() - lastActivityTime > 12000))
    {
        inMenu = false; inSensorList = false; inAddMode = false;
        LCD_WriteCommand4Bits(0x01);
    }
}

void executeAction() {
    if (currentMenuPos == 0) { systemState = ALARM_EXIT_DELAY; countdownValue = 10;
exitTimer = millis(); inMenu = false; }
    else if (currentMenuPos == 1) { systemState = ALARM_DISARMED; alarm_sense = 0;
inMenu = false; noTone(PIN_BUZZER); }
    else if (currentMenuPos == 2) { inSensorList = true; inMenu = false;
currentSensorIdx = 0; }
    else if (currentMenuPos == 3) { inAddMode = true; inMenu = false; tempPin = 4; }
}

void displayMainScreen() {
    LCD_WriteCommand4Bits(0x80);
    if (systemState == ALARM_DISARMED) LCD_WriteText("System OFF      ");
    else if (systemState == ALARM_ARMED) LCD_WriteText("Czuwanie...    ");
    else if (systemState == ALARM_TRIGGERED) LCD_WriteText("!!! ALARM !!!  ");
    else if (systemState == ALARM_EXIT_DELAY) LCD_WriteText("WYJDZ! Czas:    ");

    LCD_WriteCommand4Bits(0xC0);
    if (systemState == ALARM_EXIT_DELAY) {

```

```

        LCD_WriteData4Bits('0' + (countdownValue / 10)); LCD_WriteData4Bits('0' +
(countdownValue % 10)); LCD_WriteText(" sek          ");
    } else if (systemState == ALARM_ARMED) {
        LCD_WriteText("Ruchy: "); LCD_WriteData4Bits('0' + alarm_sense); LCD_WriteText("/3
");
    } else LCD_WriteText("OK = Menu          ");
}

```

```

void displayAddScreen() {
    LCD_WriteCommand4Bits(0x80); LCD_WriteText("WYBIERZ PIN:   ");
    LCD_WriteCommand4Bits(0xC0); LCD_WriteText("PIN: ");
    if (tempPin >= 14) {
        LCD_WriteText("A"); LCD_WriteData4Bits('0' + (tempPin - 14)); LCD_WriteText("
");
    } else {
        LCD_WriteData4Bits('0' + (tempPin / 10)); LCD_WriteData4Bits('0' + (tempPin %
10)); LCD_WriteText("          ");
    }
    LCD_WriteText("[OK]");
}

```

```

void displaySensorList() {
    LCD_WriteCommand4Bits(0x80);
    LCD_WriteText("TEST C"); LCD_WriteData4Bits('1' + currentSensorIdx);
    LCD_WriteText(sensors[currentSensorIdx].violated ? " [!] " : " [OK]");
    LCD_WriteCommand4Bits(0xC0);
    LCD_WriteText("Pin:");
    byte p = sensors[currentSensorIdx].pin;
    if (p < 14) { LCD_WriteData4Bits('0' + (p/10)); LCD_WriteData4Bits('0' + (p%10)); }
    else { LCD_WriteText("A"); LCD_WriteData4Bits('0' + (p - 14)); }
    LCD_WriteText(" OK=Wroc");
}

```

// TABELA BINARNA DLA DEKODERA CD4511

```

void update7SegmentCD4511(int state) {
    if (state == ALARM_DISARMED) { // Stan 0 -> wyświetla cyfrę 0 (000)
        digitalWrite(PIN_A, LOW);
        digitalWrite(PIN_B, LOW);
        digitalWrite(PIN_C, LOW);
    }
    else if (state == ALARM_ARMED) { // Stan 1 -> wyświetla cyfrę 1 (001)
        digitalWrite(PIN_A, HIGH);
        digitalWrite(PIN_B, LOW);
        digitalWrite(PIN_C, LOW);
    }
    else if (state == ALARM_EXIT_DELAY) { // Stan 2 -> wyświetla cyfrę 2 (010)
        digitalWrite(PIN_A, LOW);
        digitalWrite(PIN_B, HIGH);
        digitalWrite(PIN_C, LOW);
    }
    else if (state == ALARM_TRIGGERED) { // Stan 3 -> wyświetla cyfrę 3 (011)
        digitalWrite(PIN_A, HIGH);
        digitalWrite(PIN_B, HIGH);
        digitalWrite(PIN_C, LOW);
    }
    else if (state == SENSOR_VIOLATED) { // Stan 4 -> wyświetla cyfrę 4 (100)
        digitalWrite(PIN_A, LOW);
        digitalWrite(PIN_B, LOW);
    }
}

```

```

        digitalWrite(PIN_C, HIGH);
    }
}

void updateVisuals() {
    update7SegmentCD4511(systemState);

    panelLED.clear();
    if (systemState == ALARM_DISARMED) panelLED.setPixelColor(0, panelLED.Color(0, 255, 0));
    else if (systemState == ALARM_ARMED) panelLED.setPixelColor(1, panelLED.Color(0, 0, 255));
    else if (systemState == ALARM_EXIT_DELAY) panelLED.setPixelColor(2, panelLED.Color(255, 100, 0));
    else if (systemState == ALARM_TRIGGERED) panelLED.setPixelColor(3, panelLED.Color(255, 0, 0));
    else if (systemState == SENSOR_VIOLATED) panelLED.setPixelColor(4, panelLED.Color(150, 0, 255));
    panelLED.show();

    if (systemState == ALARM_TRIGGERED) {
        if (millis() - prevMillis > 250) {
            prevMillis = millis(); blink = !blink;
            digitalWrite(ledRed, blink ? LOW : HIGH); digitalWrite(ledBlue, blink ? HIGH : LOW);
            if (blink) tone(PIN_BUZZER, 1800); else tone(PIN_BUZZER, 1200);
        }
    } else if (systemState != ALARM_EXIT_DELAY) {
        digitalWrite(ledRed, HIGH); digitalWrite(ledBlue, HIGH); noTone(PIN_BUZZER);
    }
}

void displayMenu() {
    LCD_WriteCommand4Bits(0x80); LCD_WriteText("--- MENU --- ");
    LCD_WriteCommand4Bits(0xC0);
    const char* opts[] = {"> Uzbroj alarm ", "> Rozbroj alarm", "> Test czujnikow", "> Dodaj czujnik "};
    LCD_WriteText(opts[currentMenuPos]);
}

void LCD_Write4Bits(byte d) {
    digitalWrite(E, LOW); digitalWrite(DB4, d & 0x01); digitalWrite(DB5, d & 0x02);
    digitalWrite(DB6, d & 0x04); digitalWrite(DB7, d & 0x08);
    digitalWrite(E, HIGH); delay(2); digitalWrite(E, LOW); delay(2);
}

void LCD_WriteData4Bits(byte d) { digitalWrite(RS, HIGH); LCD_Write4Bits(d >> 4);
LCD_Write4Bits(d & 0x0F); }
void LCD_WriteCommand4Bits(byte d) { digitalWrite(RS, LOW); LCD_Write4Bits(d >> 4);
LCD_Write4Bits(d & 0x0F); }
void LCD_Init4bit() {
    digitalWrite(RS, LOW); digitalWrite(E, LOW); delay(20);
    LCD_Write4Bits(0x03); delay(5); LCD_Write4Bits(0x03); delay(2);
    LCD_Write4Bits(0x03); delay(2); LCD_Write4Bits(0x02);
    LCD_WriteCommand4Bits(0x28); LCD_WriteCommand4Bits(0x0C);
    LCD_WriteCommand4Bits(0x01); LCD_WriteCommand4Bits(0x06);
}
void LCD_WriteText(const char *s) { while (*s) LCD_WriteData4Bits(*s++); }

```